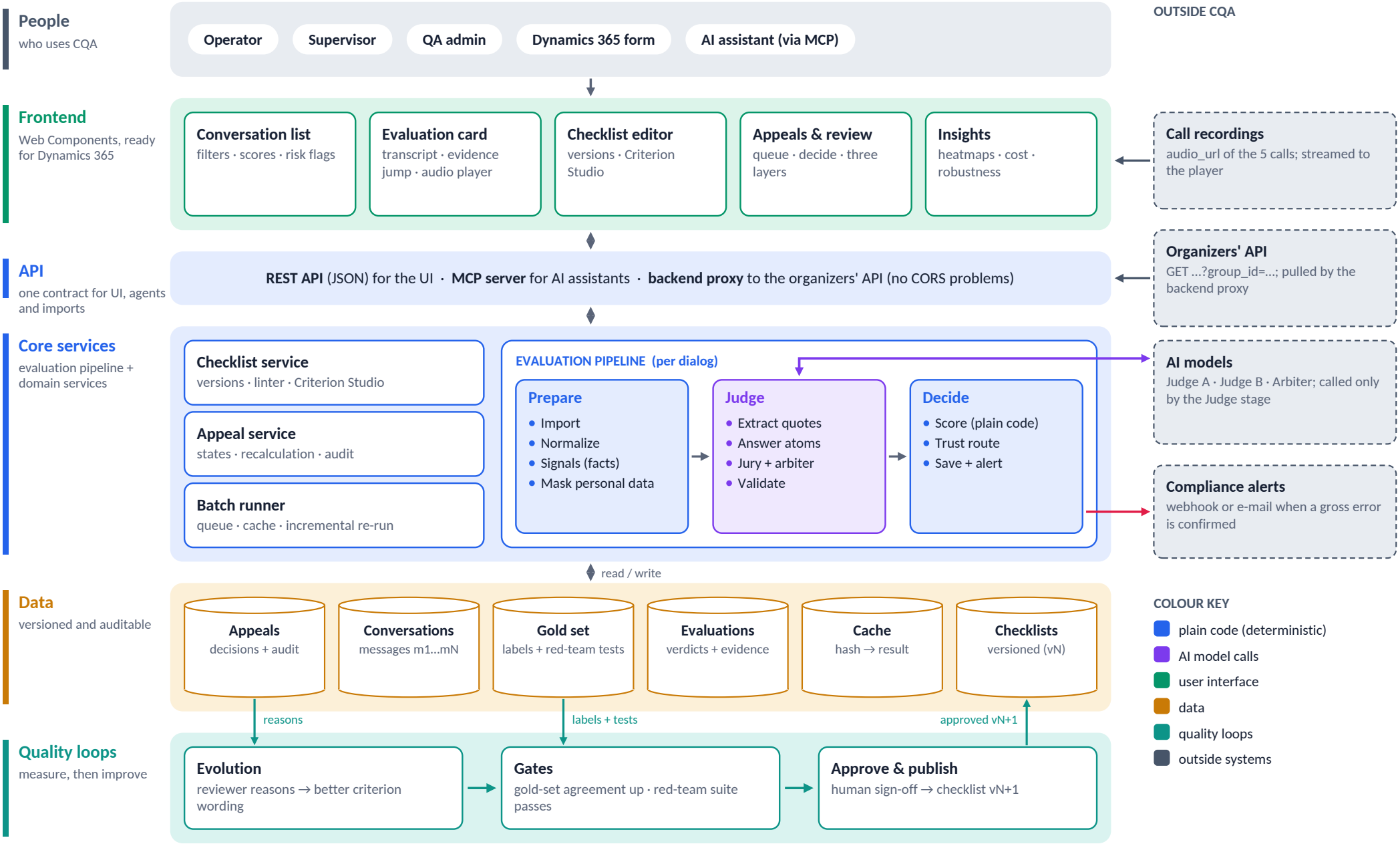
The big picture

Six layers, one rule: the AI proposes verdicts with evidence, plain code does the math, and a human always has the final word.



CQA · ARCHITECTURE

One dialog through the pipeline

Nine steps. Only steps 5 and 6 call an AI model; everything else is plain code you can unit-test.

1 Import — CODE

Fetch the set by group_id through the backend and keep the raw JSON.

```
GET …?group_id=DEV
→ 65 records
```

2 Normalize — CODE

Give every message an ID; turn MM:SS and ISO times into seconds; keep originals.

```
m7 · 00:41 → 41 s
speaker: operator
```

3 Signals — CODE

Compute the facts the judge must not guess.

```
gap m5→m6 4:12, no warning
client: KZ 70% · RU 30%
verified @ m9
```

4 Mask — CODE

Swap personal data for typed tokens before any AI call.

```
card number → [CARD_1]
IIN → [IIN_1]
name → [NAME_1]
```

5 Extract — AI

Pass 1: for every atom, quote the evidence with message IDs, or say none found.

```
A1 greeting → m3
«Сәлеметсіз бе!»
A3 intro → none, m3–m4
```

6 Judge — AI

Pass 2: answer the atoms from that evidence. Two judges vote; an arbiter breaks ties.

```
A1 ✓ A2 ✓ A3 ✗
→ 01 Contact = No
judges A + B agree
```

7 Validate — CODE

Check each quote really is in the cited message and the rules hold; retry once.

```
quote ⊂ m3 ✓
speaker = operator ✓
N/A allowed here ✓
```

8 Score — CODE

Weights, N/A redistribution, half-up rounding, gross-error flag.

```
55.5 / 80 × 100
= 69.38
```

9 Route — CODE

High confidence is auto-accepted; low confidence goes to a supervisor.

```
conf 0.93 → accept
conf 0.61 → review
```

Result

The **evaluation card** the user sees:

- verdict or scale step per criterion
- points after N/A redistribution
- clickable evidence links
- checked range for every “No”
- reason for every N/A
- total + gross-error flag
- $ per evaluation

Scoring math (plain code)

points = weight × step ÷ top step
Yes = full weight, No = 0
total = Σ points ÷ Σ applicable weights × 100
N/A criteria drop out of both sums
Round **once, at the end**, half-up to 2 decimals. Gross errors are flagged separately. No applicable weights → no total.

Trap: Python’s round(88.125, 2) gives 88.12 (banker’s rounding). Use Decimal with ROUND_HALF_UP.

Criterion	Answer	Wt	Pts
01 Contact	Yes	10	10
02 Needs	N/A	–	–
03 Identification	Yes	20	20
04 Consultation	15/20	20	15
05 Empathy	No	15	0
06 Dialog mgmt	7/10	15	10.5
07 Closing	N/A	–	–

Σ 55.5 of 80 → 69.38

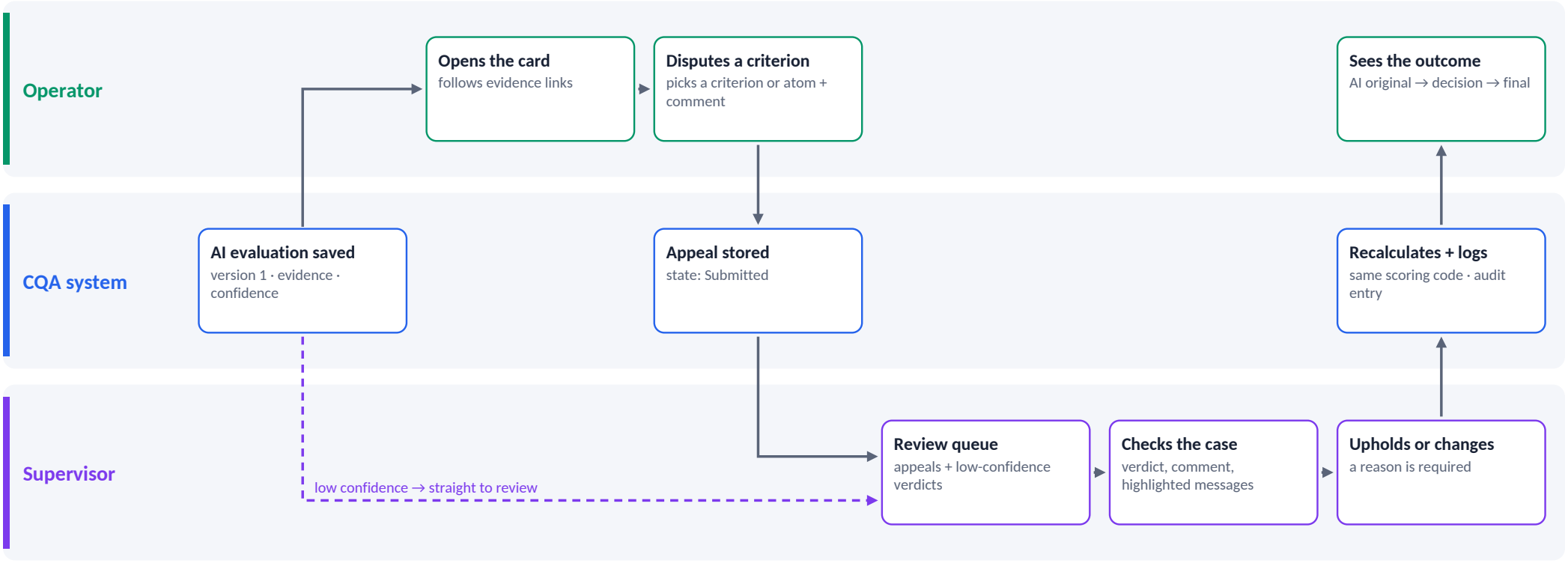
One criterion result, as stored

```
{ "criterion": "01", "verdict": "no", "points": 0,
  "atoms": { "greeting": true, "client_language": true,
             "introduced": false },
  "evidence": [{ "msg": "m3", "quote": "Сәлеметсіз бе!" }],
  "checked_range": ["m3", "m4"],
  "rationale": "Greeted in Kazakh, gave no name in the
                first reply.",
  "confidence": 0.93, "status": "auto_accepted",
  "checklist_version": 3 }
```

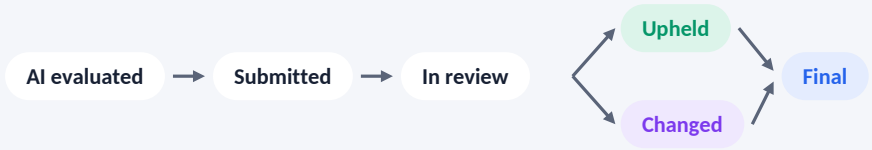
Every field is something the UI shows or the audit needs; nothing is free text the code has to parse.

Humans in the loop: appeals and review

A verdict reaches a person in two ways: the operator disputes it, or the AI was not confident enough. The human decision is final, and it is logged.



Appeal states



Every transition writes an audit event: who, when, before → after, and the reason.

One card keeps three layers

AI original
05 Empathy: **No**
total **69.38**

Reviewer decision
05 → **Yes**
“complaint acknowledged in m6”

Final
total **88.13**
recomputed by the same code

```
12:10  supervisor_B  05: no → yes  reason: complaint acknowledged in m6
```

Audit log line. Raising 05 adds 15 points: 70.5 ÷ 80 × 100 = 88.125 → 88.13.

CQA · ARCHITECTURE

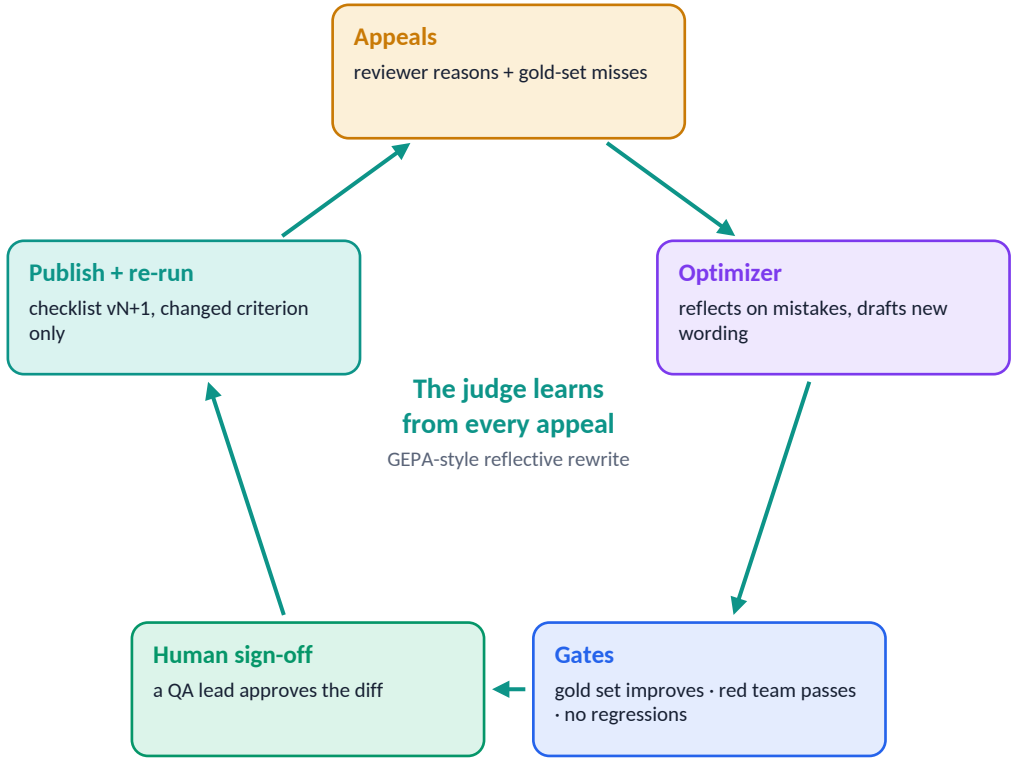
How the judge gets better, and how we know it is right

Criteria change, and every appeal is feedback. Changes go through tests and a human sign-off before they touch real scores.

CRITERION STUDIO · when a criterion is added or replaced

1. **Describe it in plain words**
   "Did the operator offer the mobile app for next time?"
2. **AI drafts the full criterion**
   definition, N/A rule, atoms, scale table, red-team tests
3. **Linter checks it**
   vague words, overlap with other criteria, weights ≠ 100
4. **You grade 5 dialogs**
   the ones the AI is least sure about; wording gets refined
5. **Publish as checklist vN+1**
   old results keep vN, so past scores stay reproducible
6. **Re-run only this criterion**
   on all dialogs, then show a before / after diff

IMPROVEMENT LOOP · every overturned verdict is feedback



TRUST INSTRUMENTS · how we know the judge is right

Gold set

15–20 hand-labelled DEV dialogs across RU, KZ and EN, including N/A and gross-error cases. Gives agreement per criterion.

Trust threshold

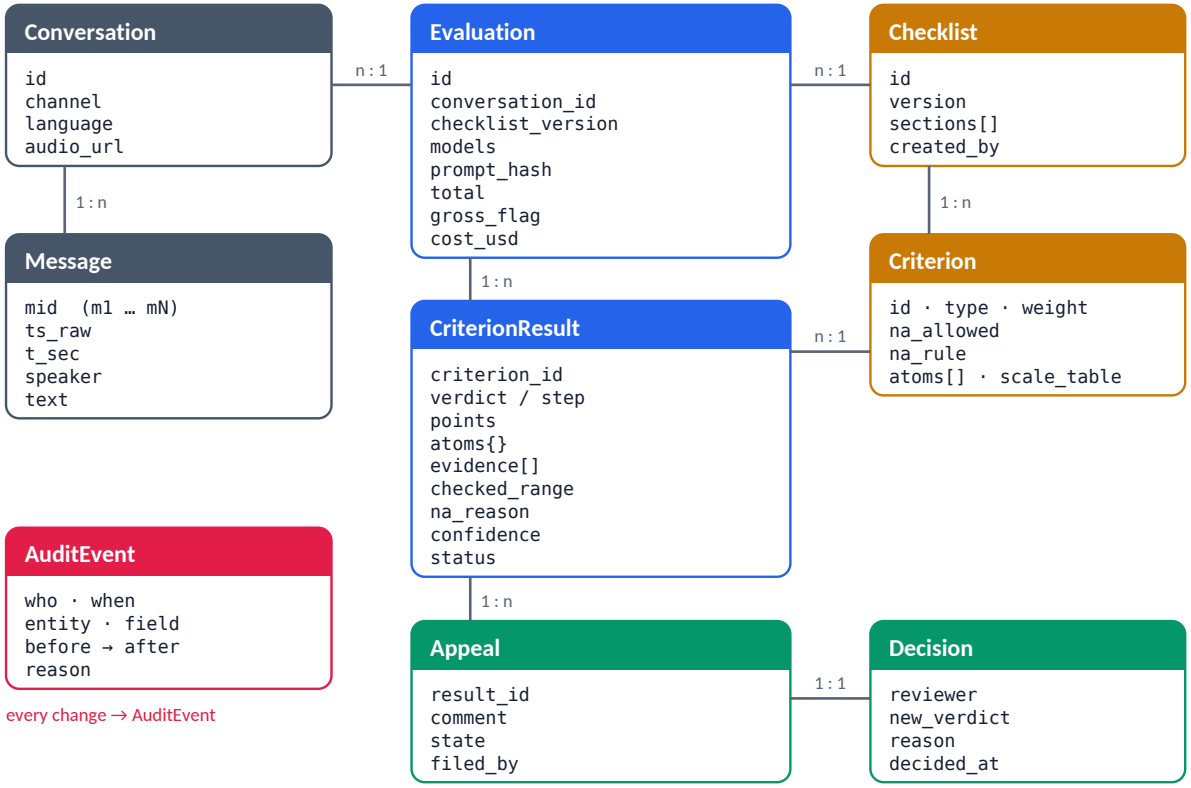
Confidence comes from jury agreement. Verdicts above the calibrated threshold are auto-accepted; the rest go to a supervisor.

Red-team suite

Edits that must flip a verdict (no greeting, late verification, a fraud offer) and harmless edits that must not.

Data model, Dynamics 365 embedding and stack

Store versions and evidence, not just scores: that is what makes appeals, audits and criterion changes cheap.



EMBEDDING IN DYNAMICS 365

Dynamics 365 form
conversation / case record
→ hosts the component in an iframe

HTML web resource
cqa_card.html?data=conv_id
→ no server code inside; bundle JS and CSS in the resource; read the record via the data parameter or parent.Xrm

CQA REST API
separate service; allow the Dynamics origin
→ all AI calls, keys and data stay server-side

AI models
judges + arbiter
→ next step: a PCF control for deeper integration

SUGGESTED STACK

Frontend	Vanilla JS Web Components, no build step
Backend	Python FastAPI + asyncio, about 8 judge calls in parallel
Storage	SQLite with JSON columns; Postgres or Dataverse later
AI	two model families behind one client; strict JSON output
Privacy	regex recognizers for IIN, cards, phones, IBAN (+ Presidio)
Tests	pytest for the scoring math and the red-team suite
Optional	MCP server · ffmpeg audio windows · Batch API for backfills

Three rules that keep this honest

- **Never overwrite a checklist.** Saving creates vN+1; each evaluation points at the version and prompt hash it used, so every past score can be reproduced.
- **Store evidence, not prose.** Message IDs, quotes and checked ranges are fields, so the UI can jump to them and the validator can test them.
- **Log every change.** Appeals, decisions and checklist edits all write an AuditEvent with who, when, before, after and why.